

PulseOps: guía para construir el proyecto

Documento para adjuntar al nuevo chat de Codex en VS Code (Ubuntu). Lee también el repositorio real antes de decidir los primeros cambios.

Mi objetivo

Soy estudiante de Ingeniería Informática y estoy aprendiendo sistemas, Cloud y DevOps. Quiero construir un proyecto propio para GitHub y mi CV, y entender qué parte disfruto más. Tengo conocimientos iniciales de Linux, Docker y Kubernetes; necesito acompañamiento para AWS, Terraform y CI/CD.

Tiempo orientativo: cuatro días para un MVP, con margen para continuar después. Enseñarme los conceptos y las razones de cada decisión es parte del objetivo.

Qué es PulseOps

Una pequeña plataforma que vigila URLs HTTP: registra servicios, los comprueba cada cierto intervalo, mide su latencia, guarda el resultado y permite consultar su estado. El valor para mi portafolio está tanto en la aplicación funcional como en la infraestructura reproducible que la ejecuta.

Alcance funcional de la primera versión

- API en Python/FastAPI: **POST /monitors** registra una URL permitida; **GET /monitors** devuelve su último estado, latencia y hora de comprobación. Añadir un endpoint de historial sencillo si cabe en el plazo.
- Worker en Python: consulta las URLs periódicamente, con tiempo de espera, registra UP/DOWN y latencia; distingue errores de red de respuestas HTTP. Definir juntos qué códigos cuentan como UP.
- PostgreSQL persiste los monitores y resultados. El MVP se usa con URLs de prueba confiables; no abrir el registro arbitrario de destinos a Internet.

Arquitectura objetivo

Desarrollo local	FastAPI + worker + PostgreSQL en Docker Compose
Infraestructura	Terraform crea red básica y una EC2 Linux en AWS; k3s corre en esa instancia
Despliegue	Imágenes Docker en un registry; API, worker y base de datos desplegados en k3s
Automatización	GitHub Actions ejecuta pruebas y publica imágenes; el primer despliegue en k3s puede ser manual

Plan de trabajo y criterios de éxito

Avanzar por capas. Al final de cada etapa, comprobar algo observable y registrar el resultado en README.

- 1 • Aplicación local** Crear API, worker, persistencia y unas pocas pruebas útiles. Demostración: registrar dos URLs, provocar una caída y consultar los resultados.
- 2 • Docker** Crear Dockerfile(s) y docker-compose.yml. Demostración: levantar todo desde cero y conservar datos al reiniciar contenedores; documentar variables de entorno.
- 3 • AWS y Terraform** Definir una VPC, subred pública, salida a Internet, reglas de red mínimas y EC2 Linux. Validar plan antes de aplicar. Comprobar acceso restringido y documentar destrucción al acabar.
- 4 • k3s** Instalar k3s y desplegar API, worker y PostgreSQL con manifiestos sencillos. Probar reinicio de pods y persistencia de datos en el nodo; explicar las limitaciones de una sola instancia.
- 5 • GitHub Actions** Crear CI para pruebas, construcción y publicación de imágenes versionadas. Comprobar un run exitoso y actualizar k3s manualmente con esa versión; describir exactamente qué parte es automática.
- 6 • Evidencias** README con arquitectura, requisitos, pasos para reproducir, pruebas, capturas o salidas representativas, costes estimados observados y decisiones técnicas. Publicar solo lo que funciona.

Definición de terminado del MVP

- Desde un clon limpio, otra persona puede ejecutar la versión local siguiendo el README.
- Las comprobaciones periódicas cambian el estado y dejan resultados consultables en PostgreSQL.
- Existe un despliegue comprobado en k3s sobre una EC2 creada con Terraform; hay una forma documentada de limpiar los recursos.
- GitHub Actions pasa las pruebas y construye/publica imágenes; el README muestra qué pasos siguen siendo manuales.

Ampliaciones después del MVP

Elegir una según interés: observabilidad con Prometheus/Grafana; Helm para despliegue reutilizable; análisis de seguridad de imágenes; o automatización del despliegue. Frontend React y alertas externas quedan para más adelante. No presentar una ampliación como terminada sin comprobarla.

Cuatro días orientativos

Día 1: aplicación y Compose. Día 2: Terraform y EC2. Día 3: k3s y CI. Día 4: pruebas de extremo a extremo, fallos, README y capturas. Si una etapa lleva más tiempo, mantener la calidad del MVP y reajustar el plan.

Instrucciones para el nuevo chat de Codex

Esta página está escrita como encargo directo para el asistente que trabaje conmigo en Ubuntu.

Quiero que seas mi tutor y colaborador técnico. Primero examina los archivos del repositorio abierto en VS Code y dime qué hay hecho y qué falta. Si el repositorio está vacío, propón una estructura mínima y empieza por el primer hito. No supongas que ya tengo código, cuenta AWS o servicios instalados.

Trabaja en pasos pequeños: explica qué vamos a hacer y por qué, implementa conmigo una pieza, enséñame cómo probarla y espera mis resultados cuando dependas de comandos ejecutados en mi Ubuntu o en AWS. Ayúdame a entender errores. Evita generar de golpe un proyecto que no sepa explicar.

Mantén una lista breve de hitos completados y pendientes. Antes de introducir una herramienta nueva, dime qué problema resuelve aquí. Cuando ya tengamos un hito comprobado, documenta lo necesario en el README y haz cambios que pueda revisar.

Decisiones y límites prácticos

- **Credenciales:** nunca guardar claves, tokens, contraseñas, kubeconfig ni terraform.tfstate en Git. Utilizar variables de entorno o secretos adecuados y .gitignore; revisar el historial antes de publicar.
- **Red y acceso:** PostgreSQL y administración de k3s sin exposición pública. Restringir SSH a mi IP o usar un acceso seguro equivalente. Si exponemos la API para la demo, limitar las rutas de escritura o protegerlas y evitar URLs hacia redes internas.
- **Costes:** antes de crear recursos AWS, estimar el coste según región, tamaño de instancia, almacenamiento e IPv4 pública. Pedirme que revise el plan y el posible gasto; al acabar, guiarme en terraform destroy y comprobar recursos residuales.
- **Alcance técnico:** una sola EC2 con k3s es un laboratorio de nodo único, no alta disponibilidad. La CI inicial termina en el registry; llamar CD al flujo solo cuando haya despliegue automatizado comprobado.

Qué debe quedar en GitHub y en el CV

Repositorio con código, pruebas, Dockerfiles, Compose, Terraform, manifiestos k3s y workflow; README reproducible con diagrama sencillo y pruebas reales. En el CV describir solo funcionalidades verificadas: plataforma de comprobación HTTP, infraestructura aprovisionada, despliegue en k3s y automatizaciones que hayan funcionado.

Primer paso al abrir el chat

“Lee este documento y revisa la carpeta que tengo abierta. Resúmeme el estado real del proyecto en cinco líneas y propón el primer paso ejecutable para el día 1. Después acompáñame paso a paso; quiero aprender mientras lo construyo.”